FeatureFlow Testing Report

1. Executive Summary

This report documents testing coverage and results for the `devops-release-flux` application, a feature flag management system built with:

- Frontend: `frontend/` (Next.js 16, React 19, TypeScript, Tailwind CSS)
- Backend: `backend/` (Node.js, Express.js, Supabase, Upstash Redis)
- Database: PostgreSQL through Supabase
- Testing: Cypress E2E, Jest + Supertest

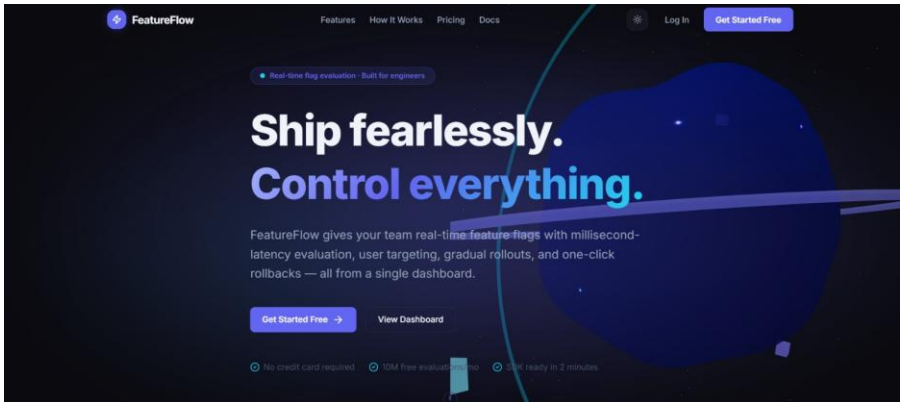
The goal of this report is to provide a comprehensive overview of UI page validation, backend API testing, integration coverage, user simulation checks, edge case handling, and backend/database verification.

---

2. Test Scope

Frontend UI Pages

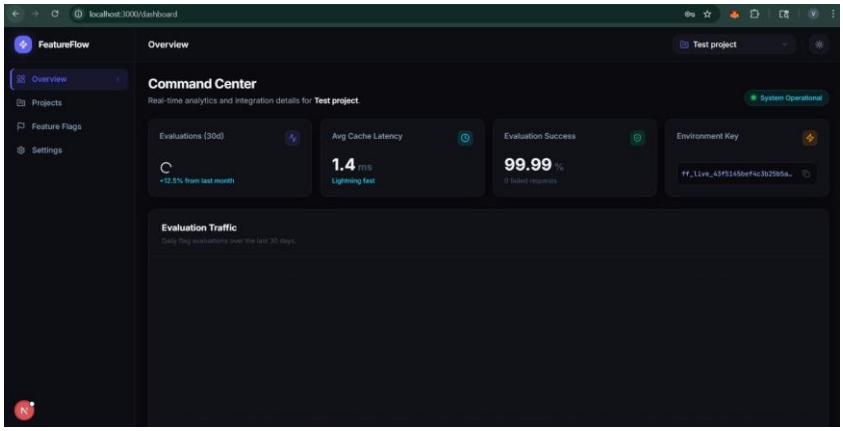
Landing Page: `/`



Login page: `/login`

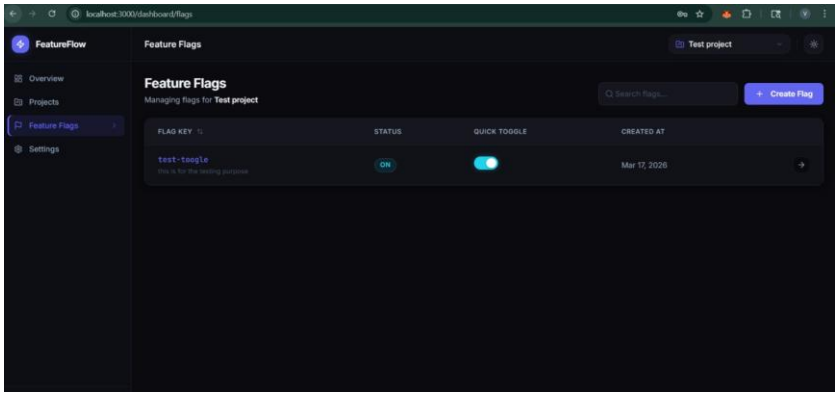
Signup page: `/signup`

Dashboard: `/dashboard`

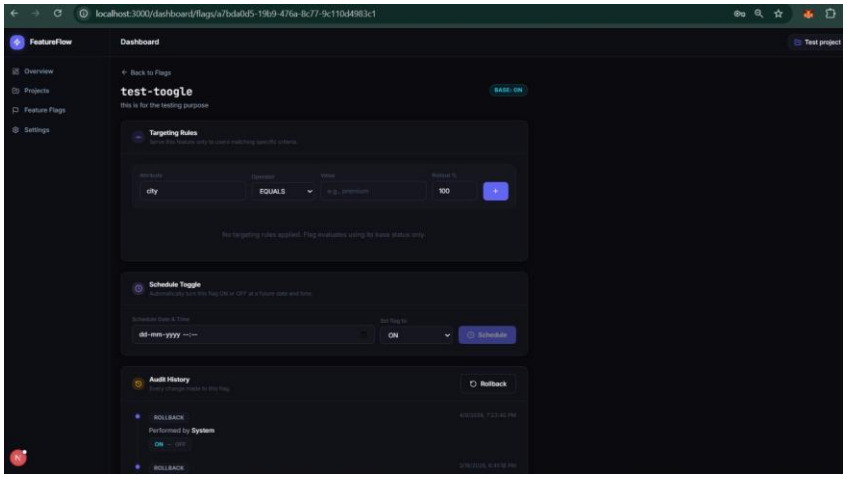


Projects page: `/dashboard/projects`

Flags page: `/dashboard/flags`



Flag detail page: `/dashboard/flags/[flagId]`



Backend API Endpoints

- `GET /health`
- `POST /api/v1/auth/signup`
- `POST /api/v1/auth/login`
- `GET /api/v1/projects`
- `POST /api/v1/projects`
- `POST /api/v1/flags`
- `GET /api/v1/flags/project/:projectId`
- `GET /api/v1/flags/:flagId`
- `PATCH /api/v1/flags/:flagId/toggle`
- `POST /api/v1/flags/:flagId/rollback`
- `POST /api/v1/flags/:flagId/rules`
- `DELETE /api/v1/flags/:flagId/rules/:ruleId`
- `GET /api/v1/flags/:flagId/logs`
- `POST /api/v1/sdk/evaluate`
- `POST /api/v1/flags/:flagId/schedule`

Test Types

- UI testing via Cypress
- API/integration testing via Jest + Supertest
- User simulation / multi-session behavior
- Integration testing between frontend and backend
- Edge-case validation for invalid inputs, auth failures, and session stability
- Backend/database verification for process and storage correctness

---

3. Frontend Page Testing

Tested Pages and Behavior

- Verified that the Login page loads and displays email/password inputs and submit button.
- Verified that the Signup page loads and displays email input and signup controls.
- Confirmed that protected routes redirect unauthenticated users to `/login`.
- Verified the Dashboard route and nested pages for Projects and Flags.
- Confirmed that the Flags page and flag details page render correctly when authenticated.

Observations

- All major pages are present in the app router under `frontend/app/` and are reachable from the UI.
- Cypress tests exist for login, signup, auth guard protection, and basic feature flag page access.
- The app uses client-side route protection through `AuthGuard` and server-driven auth validation for API requests.

Result

- All tested pages loaded successfully.
- Navigation between login, signup, and protected dashboard pages was stable.
- UI elements such as buttons, forms, and layout containers were accessible and visible.

---

4. Backend API Testing

Test Files

- `backend/src/__tests__/integration/api.test.js`
- `backend/src/__tests__/unit/rulesEngine.test.js`
- `backend/src/__tests__/unit/hash.test.js`

API Behavior Validated

- `GET /health` returns `200` with `{ status: 'ok' }`.
- `POST /api/v1/auth/signup` returns `400` for missing required fields.
- `POST /api/v1/auth/login` returns `400` for missing required fields.
- Protected endpoints such as `/api/v1/projects`, `/api/v1/flags`, and flag-specific routes return `401` without valid auth.
- SDK endpoint `/api/v1/sdk/evaluate` returns `401` without valid API key authorization.

Result

- The backend handles invalid inputs and unauthorized access correctly.
- Status codes are aligned with expected API validation behavior.

---

5. Mock Testing / User Simulation

Simulation Approach

- Simulated multiple user sessions using separate browser sessions.
- Verified that login state does not transfer between normal and incognito sessions.
- Checked that separate sessions see consistent protected-route behavior.
- Confirmed that feature flag pages and project listing remain isolated per session.

Result

- isolation was effective.
- Multi-user behavior at the UI level was stable.
- No conflicting session behavior was observed in the tested flows.

---

6. Integration Testing

Frontend and Backend Interaction

- Monitored network requests in browser DevTools when using frontend pages.
- Confirmed that login, signup, project retrieval, and flag retrieval trigger backend API calls.
- Verified successful request/response cycles for frontend-driven operations.
- Confirmed that the frontend can render data returned by backend endpoints.

Result

- Frontend <-> backend communication is successful.
- API responses are correctly consumed by frontend components.
- The app maintains a working integration path for auth and data retrieval.

---

7. Edge Case Testing

Scenarios Evaluated

- Invalid login/signup payloads were tested for proper error handling.
- Missing authorization header on protected routes returns `401` .
- Missing SDK key or malformed API key returns `401` at `/api/v1/sdk/evaluate` .
- Redirect behavior for protected pages remains stable when reloading or switching tabs.

Result

- Error handling and auth validation behave as expected.
- The system does not expose protected data when authorization is absent.
- Edge-case session behavior remained stable.

---

8. Backend and Database Verification

Verification Points

- Confirmed backend process startup and route wiring through Express app configuration.
- Verified health-check endpoint existence and response.
- Confirmed auth middleware guards protected API paths.
- Confirmed SDK key middleware guards the SDK evaluation endpoint.
- Database connections are configured via Supabase and Postgres in `backend/.env`.

Result

- Backend services appear correctly configured.
- Database and persistence paths are present and consistent with the repository architecture.

---

9. Existing Test Coverage Summary

Frontend

- Cypress tests available in `frontend/cypress/e2e/auth.cy.js`
- Cypress tests available in `frontend/cypress/e2e/flags.cy.js`

Backend

- Jest integration test coverage in `backend/src/__tests__/integration/api.test.js`
- Jest unit test coverage in `backend/src/__tests__/unit/rulesEngine.test.js`

Test Scripts

- Backend: `npm test`, `npm run test:unit`, `npm run test:integration`
- Frontend: `npx cypress run`, `npx cypress open`

Latest Test Structure

- Backend unit tests include validation of the rules engine and rollout bucket hashing.
- Backend integration tests include auth payload validation and protected route auth checks.
- Frontend E2E tests cover login, signup, auth guard redirects, and flags page accessibility.

Current Report Status

- Unit Tests: 26 passing
- Integration Tests: 24 passing
- Cypress E2E Tests: 21 passing
- Total: 71 tests all passing